Normalisasi Data (1NF 2NF)

Oleh:
Ir. Hira Laksmiwati Soemitro, M.Sc
Hartanto Tantriawan, S.Kom., M.Kom
Arief Ichwani, S.Kom., M.Cs

Program Studi Teknik Informatika
Institut Teknologi Sumatera
2026

Capaian pembelajaran

- Menentukan ketergantungan fungsional (***functional dependency***) antara dua atau lebih atribut yang merupakan bagian dari suatu relasi.
- Menghubungkan konstrain yang dinyatakan sebagai ***primary key*** dan ***foreign key*** dengan ketergantungan fungsional.
- Menghitung ***closure*** dari sekumpulan atribut berdasarkan ketergantungan fungsional yang diberikan.
- Menentukan apakah sekumpulan atribut membentuk ***superkey*** dan/atau ***candidate key*** untuk suatu relasi dengan ketergantungan fungsional yang diberikan.
- Mengevaluasi usulan dekomposisi untuk menentukan apakah dekomposisi tersebut bersifat ***lossless-join*** (penggabungan tanpa kehilangan data) dan ***dependency-preservation*** (pemeliharaan ketergantungan).
- Mendeskripsikan definisi dari **1NF, 2NF, 3NF, dan BCNF.**
- Mengidentifikasi apakah suatu relasi memenuhi kriteria **1NF, 2NF, 3NF, atau BCNF.**
- Melakukan normalisasi relasi **1NF** menjadi sekumpulan relasi **3NF** (atau **BCNF**) dan melakukan denormalisasi pada skema relasional.
- Menjelaskan dampak normalisasi terhadap efisiensi operasi basis data, khususnya pada optimasi kueri (***query optimization***).

Kombinasi Skema

- Asumsikan kita menggabungkan skema ***instructor* (instruktur)** dan ***department* (departemen)** menjadi satu skema ***inst_dept.***
- Hasilnya adalah kemungkinan terjadinya **repetisi** (pengulangan) informasi.

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Skema Gabungan Tanpa Redudansi

- Pertimbangkan penggabungan **dua relasi** berikut:
  - **sec_class** (*sec_id, building, room_number*) dan
  - **section** (*course_id, sec_id, semester, year*)
- Menjadi **satu relasi tunggal**:
  - **section** (*course_id, sec_id, semester, year, building, room_number*)
- Dalam kasus ini, tidak terjadi redundansi informasi.

Kelemahan dalam Perancangan Basis Data Relasional

- **Perancangan basis data relasional mengharuskan kita untuk menemukan kumpulan skema relasi yang "baik". Perancangan yang buruk dapat menyebabkan:**
  - Repetisi (pengulangan) informasi.
  - Ketidakmampuan untuk merepresentasikan informasi tertentu.
- **Contoh:**
  - ***Lending-schema*** = (*branch-name, branch-city, assets, customer-name, loan-number, amount*).

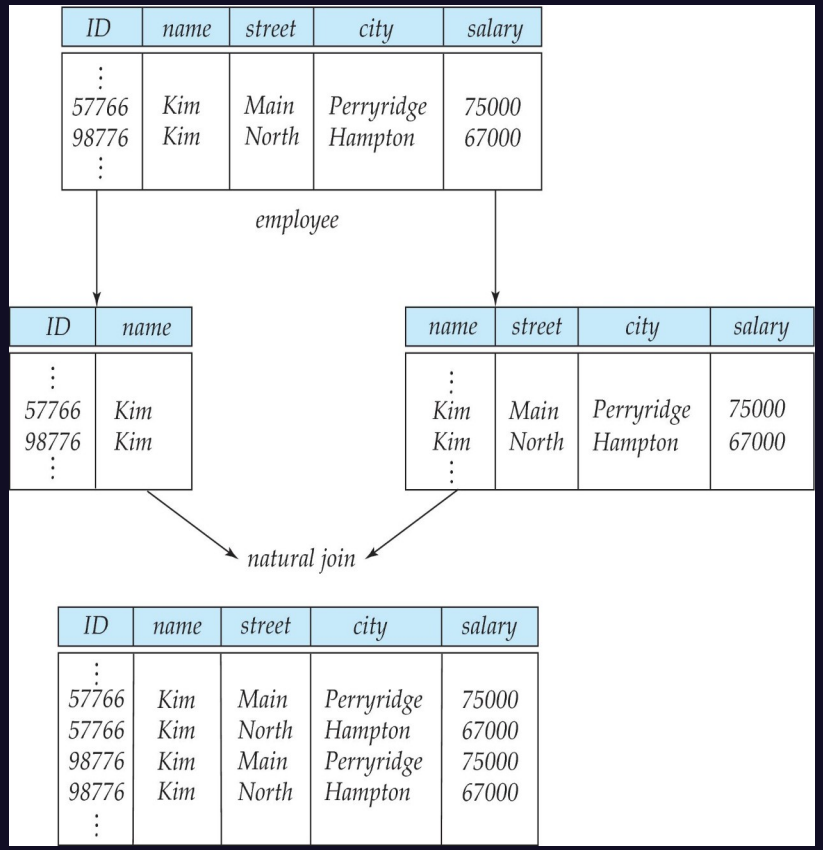
branch-name	*branch-city*	*assets*	*customer-name*	*loan-number*	*amount*
Downtown	Brooklyn	9000000	Jones	L-17	1000
Redwood	Palo Alto	2100000	Smith	L-23	2000
Perryridge	Horseneck	1700000	Hayes	L-15	1500
Downtown	Brooklyn	9000000	Jackson	L-14	1500

- **Tujuan Perancangan:**
  - Menghindari redundansi data.
  - Memastikan hubungan antar atribut terepresentasi dengan baik.
  - Memfasilitasi pemeriksaan pembaruan (*updates*) terhadap pelanggaran konstrain integritas basis data.

Bagaimana dengan Skema yang Lebih Kecil?

- Andaikata kita memulai dengan skema ***inst_dept***. Bagaimana kita mengetahui cara memecah (mendekomposisi) skema tersebut menjadi skema ***instructor*** dan ***department***?
- Buatlah sebuah aturan: "jika terdapat skema (***dept_name, building, budget***), maka ***dept_name*** akan menjadi ***candidate key*** (kunci kandidat)."
- Dinotasikan sebagai ketergantungan fungsional (***functional dependency***):
  - ***dept_name → building, budget***
- Dalam skema ***inst_dept***, karena ***dept_name*** **bukan** merupakan ***candidate key***, maka data gedung (***building***) dan anggaran (***budget***) suatu departemen mungkin harus dituliskan berulang kali.
  - Hal ini mengindikasikan perlunya dilakukan dekomposisi terhadap skema *inst_dept*.
- **Tidak** semua dekomposisi bersifat baik. Andaikan kita mendekomposisi *employee(ID, name, street, city, salary)* menjadi:
  - *employee1 (ID, name)*
  - *employee2 (name, street, city, salary)*
- Slide berikutnya akan menunjukkan bagaimana kita kehilangan informasi — kita tidak dapat merekonstruksi relasi *employee* yang asli — sehingga dekomposisi ini disebut sebagai *lossy decomposition* (dekomposisi yang menyebabkan kehilangan informasi).

Lossy Decomposition



Contoh dari Lossless-Join Decomposition

- Dekomposisi *lossless join* (penggabungan tanpa kehilangan data)
- Dekomposisi dari $R = (A, B, C)$

  $R_1 = (A, B)$ $R_2 = (B, C)$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$$\Pi_A(r) \bowtie \Pi_B(r)$$

A	B	C
α	1	A
β	2	B

1st normal form

- Domain bersifat atomik jika elemen-elemen di dalamnya dianggap sebagai unit yang tidak dapat dibagi lagi (***indivisible units***).
  - Contoh domain non-atomik:
    - Kumpulan nama, atribut komposit.
    - Nomor identifikasi seperti **CS101** yang dapat dipecah menjadi beberapa bagian.
- Sebuah skema relasional R berada dalam bentuk normal pertama (***first normal form***) jika domain dari semua atribut dalam R bersifat atomik.
- Nilai-nilai non-atomik mempersulit penyimpanan dan memicu redundansi (pengulangan) penyimpanan data.
  - Contoh: Kumpulan akun yang disimpan untuk setiap pelanggan, dan kumpulan pemilik yang disimpan untuk setiap akun.
  - Kita berasumsi bahwa semua relasi berada dalam bentuk normal pertama.

1st normal form (Continue)

Atomisitas sebenarnya merupakan properti yang bergantung pada bagaimana elemen-elemen dari suatu domain digunakan.

- **Contoh:** String biasanya dianggap sebagai unit yang tidak dapat dibagi.
- **Andaikan mahasiswa diberikan nomor induk (roll numbers) berupa string dengan format *CS0012* atau *EE1127*.**
- **Jika dua karakter pertama diekstraksi untuk menentukan departemen, maka domain dari nomor induk tersebut tidak bersifat atomik.**
- **Melakukan hal tersebut merupakan tindakan yang kurang tepat karena menyebabkan terjadinya penyandian informasi (*encoding of information*) di dalam program aplikasi, alih-alih di dalam basis data itu sendiri.**

Tujuan(Goal) – Menyusun teori untuk hal-hal berikut

- **Menentukan** apakah suatu relasi tertentu R berada dalam bentuk yang "baik".
- **Dalam hal** sebuah relasi R tidak berada dalam bentuk yang "baik", **dekomposisikan** (pecah) relasi tersebut ke dalam sekumpulan relasi $\{R_1, R_2, ..., R_n\}$ sedemikian rupa sehingga:
  - setiap relasi berada dalam bentuk yang baik
  - dekomposisinya merupakan *lossless-join decomposition* (dekomposisi gabungan tanpa kehilangan data)
- **Teori kita didasarkan pada:**
  - ketergantungan fungsional (*functional dependencies*)
  - ketergantungan bernilai banyak (*multivalued dependencies*)

Ketergantungan Fungsional (Functional Depedencies)

- **Batasan-batasan** pada kumpulan relasi yang sah.
- **Mengharuskan** nilai untuk sekumpulan atribut tertentu menentukan secara unik nilai untuk sekumpulan atribut lainnya.
- Ketergantungan fungsional adalah sebuah **generalisasi** dari konsep sebuah *key* (kunci).

Ketergantungan Fungsional (Continue)

- Misalkan R adalah sebuah skema relasi
  - $\alpha \subseteq R$ *and* $\beta \subseteq R$
- Ketergantungan fungsional
  - $\alpha \rightarrow \beta$

  berlaku pada R jika dan hanya jika untuk setiap relasi yang sah *r(R)*, kapan pun ada dua tupel t_1 dan t_2 dalam r memiliki nilai yang sama pada atribut α, mereka juga memiliki nilai yang sama pada atribut *β*. Yaitu:

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- **Contoh:** Perhatikan *r*(A,*B*) dengan instans r berikut :

1	4
1	
	5
3	
	7

- Pada instans ini, *A* → *B* **TIDAK** berlaku, tetapi *B* → *A* berlaku.

Ketergantungan Fungsional (Continue)

- *K* adalah sebuah **superkey** untuk skema relasi *R* jika dan hanya jika $K \rightarrow R$.
- *K* adalah sebuah **candidate key** untuk *R* jika dan hanya jika
  - $K \rightarrow R$, dan
  - tidak ada $\alpha \subset K$ sedemikian rupa sehingga $\boldsymbol{\alpha \rightarrow R}$.
- Ketergantungan fungsional memungkinkan kita untuk menyatakan batasan-batasan (***constraints***) yang <u>tidak dapat dinyatakan</u> menggunakan ***superkeys***.

- Perhatikan skema berikut: **inst_dept** (ID, name, salary, dept_name, building, budget) Kita mengharapkan ketergantungan fungsional ini berlaku:

$$dept_name \rightarrow building$$

dan

$$ID \rightarrow building$$

tetapi **tidak** mengharapkan hal berikut berlaku:

$$dept_name \rightarrow salary$$

Penggunaan Ketergantungan Fungsional

- **Kita menggunakan ketergantungan fungsional untuk:**
  - Menguji relasi untuk melihat apakah mereka **sah (*legal*)** di bawah sekumpulan ketergantungan fungsional yang diberikan.
    - Jika sebuah relasi r sah di bawah sekumpulan ketergantungan fungsional F, kita katakan bahwa r **memenuhi** (***satisfies***) F.
  - **Menentukan batasan (*constraints*)** pada kumpulan relasi yang sah.
    - Kita katakan bahwa F **berlaku (*holds*)** pada R jika semua relasi yang sah pada R memenuhi kumpulan ketergantungan fungsional F.
- **Catatan**: Sebuah *instance* spesifik dari skema relasi mungkin memenuhi suatu ketergantungan fungsional meskipun ketergantungan fungsional tersebut tidak berlaku pada semua instans yang sah.
  - Sebagai contoh, sebuah *instance* spesifik dari tabel instructor mungkin saja, secara kebetulan, memenuhi:

$$name \rightarrow ID$$

Contoh

ID	Nama	Alamat
001	Budi	Jakarta
002	Siti	Bandung
003	Agus	Surabaya

Jika kita melihat data (instance) pada tabel, seolah-olah berlaku:

- ***Nama → ID*** *: benar untuk data ini, karena setiap nama merujuk ke ID yang unik*
- ***Alamat → Nama*** *: benar untuk data ini, karena setiap Alamat hanya punya satu nama.*

Namun secara aturan system (skema), kita **TIDAK BOLEH** menetapkan sebagai ***Nama → ID*** aturan baku. Misalnya pada baris selanjutnya ada nama Budi dengan ID 004. hal ini akan melanggar aturan ***Nama → ID.*** Kegagal aturan ini terjadi karena pada kolom nama terdapat dua nama Budi, dengan ID yang berbeda.

Ketergantungan Fungsional (Continue)

Sebuah ketergantungan fungsional dikatakan **trivial** jika ia dipenuhi oleh semua instans dari suatu relasi.

- **Contoh:**
  - *ID, name* $\rightarrow$ *ID*
  - *name* $\rightarrow$ *name*
- **Secara umum,** $\alpha \rightarrow \beta$ **adalah trivial jika** $\beta \subseteq \alpha$

Teori Ketergantungan-Fungsional

- **Kita sekarang mempertimbangkan** teori formal yang memberi tahu kita ketergantungan fungsional mana yang tersirat secara logis oleh sekumpulan ketergantungan fungsional yang diberikan.
- **Kita kemudian mengembangkan** algoritma untuk menghasilkan dekomposisi tanpa kehilangan data (*lossless decompositions*) ke dalam bentuk **BCNF** dan **3NF**.
- **Kita kemudian mengembangkan** algoritma untuk menguji apakah suatu dekomposisi bersifat mempertahankan ketergantungan (*dependency-preserving*).

Penutupan (*Closure*) dari Sekumpulan Ketergantungan Fungsional

- **Diberikan** sekumpulan *F* ketergantungan fungsional, terdapat ketergantungan fungsional tertentu lainnya yang secara logis tersirat oleh *F*.
  - **Sebagai contoh:** Jika $A \rightarrow B$ dan $B \rightarrow C$, maka kita dapat menyimpulkan bahwa $A \rightarrow C$.
- Sekumpulan dari **semua** ketergantungan fungsional yang secara logis tersirat oleh *F* disebut sebagai **penutupan** (***closure***) dari *F*.
- Kita melambangkan ***closure*** dari *F* dengan F^+.
- F^+ adalah sebuah **superset** dari *F*.

Penutupan (*Closure*) dari Sekumpulan Ketergantungan Fungsional

- **Kita dapat menemukan F^+**, yaitu *closure* dari F, dengan menerapkan secara berulang **Aksioma Armstrong**:
  - jika $\beta \subseteq \alpha$, maka $\alpha \rightarrow \beta$ (**reflexivity**)
  - jika $\alpha \rightarrow \beta$, maka $\gamma\alpha \rightarrow \gamma\beta$ (**augmentation**)
  - jika $\alpha \rightarrow \beta$, dan $\beta \rightarrow \gamma$, maka $\alpha \rightarrow \gamma$ (**transitivity**)
- **Aturan-aturan ini bersifat:**
  - ***sound*** (menghasilkan hanya ketergantungan fungsional yang benar-benar berlaku), dan
  - ***complete*** (menghasilkan semua ketergantungan fungsional yang berlaku).

Contoh

- Misalkan kita memiliki sebuah tabel dengan atribut {A, B, C} dan sekumpulan aturan dasar F sebagai berikut:
  - $A \rightarrow B$
  - $B \rightarrow C$
- Sekarang kita terapkan **Aksioma Armstrong** untuk mencari semua aturan yang mungkin (isi dari F^+):
  1. **Menggunakan Transitivitas (Transitivity)**
     - Karena kita tahu $A \rightarrow B$ dan $B \rightarrow C$, maka menurut aturan **transitivitas**, kita bisa menyimpulkan:
       - $A \rightarrow C$ (Aturan baru 1)
  2. **Menggunakan Augmentasi (Augmentation)**
     - Kita bisa menambahkan atribut apa saja di kedua sisi. Misalnya, jika kita tambahkan A pada aturan $B \rightarrow C$, maka:
     - $AB \rightarrow AC$ (Aturan baru 2)
  3. **Menggunakan Refleksivitas (Reflexivity).** Ini adalah aturan yang "pasti benar" (**trivial**). Karena A adalah bagian dari {A, B}, maka:
     - $AB \rightarrow A$
     - $AB \rightarrow B$
     - $A \rightarrow A$

Contoh (lanjutan)

- **Hasil Akhir F^+**
- Maka, F^+ berisi semua aturan asli ditambah semua aturan baru yang kita temukan tadi:

  $F^+ = \{A \rightarrow B, B \rightarrow C, A \rightarrow C, AB \rightarrow AC, AB \rightarrow A$,...dan seterusnya$\}$

- **Kenapa ini penting?**
- Dalam desain database, mengetahui $A \rightarrow C$ sangat penting karena ini menunjukkan adanya **ketergantungan transitif**. Jika kita membiarkan $A \rightarrow B \rightarrow C$ dalam satu tabel, biasanya akan terjadi redundansi (data berulang) yang tidak efisien. Itulah alasan kita perlu melakukan **Normalisasi**.

Contoh (lanjutan)

- $R = (A, B, C, G, H, I)$
  $F = \{ A \rightarrow B$
  $A \rightarrow C$
  $CG \rightarrow H$
  $CG \rightarrow I$
  $B \rightarrow H\}$
- some members of F^+
  - $A \rightarrow H$
    - berdasarkan **transivitas** dari $A \rightarrow B$ *dan* $B \rightarrow H$
  - $AG \rightarrow I$
    - dengan **meng-augmentasi** $A \rightarrow C$ dengan G, untuk memperoleh $\boldsymbol{AG \rightarrow CG}$ dan kemudian transivitas dengan $\boldsymbol{CG \rightarrow I}$
  - $CG \rightarrow HI$
    - dengan **meng-augmentasi** $CG \rightarrow I$ untuk menyimpulkan $CG \rightarrow CGI$, dan **meng-augmentasi** $CG \rightarrow H$ untuk menyimpulkan $CGI \rightarrow HI$, dan kemudian **Transivitas**

Prosedur untuk Menghitung F^+

- **Untuk menghitung penutupan (*closure*) dari sekumpulan ketergantungan fungsional F:**

F^+ = F
repeat
for each ketergantungan fungsional ***f*** di dalam ***F^+***
terapkan aturan refleksivitas (*reflexivity*) dan augmentasi (*augmentation*) pada ***f***. tambahkan hasil ketergantungan fungsional tersebut ke ***F^+***
for each pasangan ketergantungan fungsional f_1 dan f_2 di dalam ***F^+***
if f_1 dan f_2 dapat digabungkan menggunakan **transitivitas**
then tambahkan hasil ketergantungan fungsional tersebut ke ***F^+***
until F^+ tidak berubah lagi

CATATAN: Kita akan melihat prosedur alternatif untuk tugas ini nanti.

Penutupan (*Closure*) dari Sekumpulan Ketergantungan Fungsional

- **Aturan tambahan:**
  - Jika $\alpha \rightarrow \beta$ berlaku dan $\alpha \rightarrow \gamma$ berlaku, maka $\alpha \rightarrow \beta\gamma$ berlaku (**gabungan** / ***union***)
  - Jika $\alpha \rightarrow \beta\gamma$ berlaku, maka $\alpha \rightarrow \beta$ berlaku dan $\alpha \rightarrow \gamma$ berlaku (**dekomposisi** / ***decomposition***)
  - Jika $\alpha \rightarrow \beta$ berlaku dan $\beta\gamma \rightarrow \delta$ berlaku, maka $\alpha\gamma \rightarrow \delta$ berlaku (**pseudotransitivitas** / ***pseudotransitivity***)
- **Aturan-aturan di atas dapat disimpulkan dari aksioma-aksioma Armstrong.**

Penutupan dari Kumpulan Atribut

- **Diberikan** sekumpulan atribut α, definisikan **penutupan** (***closure***) dari α di bawah F (dilambangkan dengan α^+) sebagai sekumpulan atribut yang ditentukan secara fungsional oleh α di bawah F.
- **Algoritma untuk menghitung α^+, yaitu *closure* dari α di bawah F:**

```
result := α;
        while (ada perubahan pada result) do
                for each β → γ in F do
                begin
                if β ⊆ result then result := result ∪ γ
end
```

Example of Attribute Set Closure

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B$
  $A \rightarrow C$
  $CG \rightarrow H$
  $CG \rightarrow I$
  $B \rightarrow H\}$
- $(AG)^+$
  1. $result = AG$
  2. $result = ABCG$ $(A \rightarrow C$ and $A \rightarrow B)$
  3. $result = ABCGH$ $(CG \rightarrow H$ and $CG \subseteq AGBC)$
  4. $result = ABCGHI$ $(CG \rightarrow I$ and $CG \subseteq AGBCH)$
- Is AG a candidate key?
  1. Is AG a super key?
     1. Does $AG \rightarrow R$? == Is $(AG)^+ \supseteq R$
  2. Is any subset of AG a superkey?
     1. Does $A \rightarrow R$? == Is $(A)^+ \supseteq R$
     2. Does $G \rightarrow R$? == Is $(G)^+ \supseteq R$

Kegunaan dari Penutupan Atribut (Attribute Closure)

- Terdapat beberapa kegunaan dari algoritma penutupan atribut (*attribute closure*):
- **Menguji *superkey*:**
  - Untuk menguji apakah α adalah sebuah *superkey*, kita menghitung α^+ dan memeriksa apakah α^+ berisi semua atribut yang ada pada R.
- **Menguji ketergantungan fungsional:**
  - Untuk memeriksa apakah sebuah ketergantungan fungsional $\alpha \rightarrow \beta$ berlaku (atau dengan kata lain, ada di dalam F^+), cukup periksa apakah $\beta \subseteq \alpha^+$.
  - Artinya, kita menghitung α^+ dengan menggunakan penutupan atribut, lalu memeriksa apakah hasilnya mengandung β.
  - Ini adalah pengujian yang sederhana, murah (secara komputasi), dan sangat berguna.
- **Menghitung penutupan dari F :**
  - Untuk setiap $\gamma \subseteq R$, kita mencari penutupan γ^+, dan untuk setiap $S \subseteq \gamma^+$, kita mengeluarkan hasil berupa ketergantungan fungsional $\gamma \rightarrow S$.

Penutup Kanonik (*Canonical Cover*)

- Himpunan ketergantungan fungsional mungkin memiliki ketergantungan yang redundan (berlebihan) yang dapat disimpulkan dari ketergantungan lainnya.
  - Sebagai contoh: $A \rightarrow C$ adalah redundan dalam: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
  - Bagian-bagian dari suatu ketergantungan fungsional mungkin bersifat redundan:
    - Contoh pada sisi kanan (RHS):
      $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ dapat disederhanakan menjadi
      $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
    - Contoh pada sisi kiri (LHS):
      $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ dapat disederhanakan menjadi
      $\{A \rightarrow B, B \rightarrow C, A \rightarrow D)$
- Secara intuitif, penutup kanonik dari *F* adalah himpunan ketergantungan fungsional yang "minimal" yang setara dengan *F*, di mana tidak terdapat ketergantungan yang redundan maupun bagian-bagian ketergantungan yang redundan.

Extraneous Attributes

- Pertimbangkan sebuah himpunan *F* dari ketergantungan fungsional (*functional dependencies*) dan ketergantungan fungsional $\alpha \rightarrow \beta$ di dalam *F*.
  - Atribut A adalah **ekstranus** di dalam α jika $A \in \alpha$
    dan *F* secara logis menyiratkan $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
  - Atribut A adalah **ekstranus** di dalam β jika $A \in \beta$
    dan himpunan ketergantungan fungsional
    $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ secara logis menyiratkan *F*.
- **Catatan:** Implikasi ke arah sebaliknya bersifat **trivial** (sudah pasti benar) pada masing-masing kasus di atas, karena ketergantungan fungsional yang "lebih kuat" akan selalu menyiratkan yang "lebih lemah".

- **Contoh:** Diberikan $F = \{A \rightarrow C, AB \rightarrow C\}$
  - B adalah ekstranus dalam $AB \rightarrow C$ karena $\{A \rightarrow C, AB \rightarrow C\}$ secara logis menyiratkan
    $A \rightarrow C$ (yaitu hasil dari menghapus B dari $AB \rightarrow C$).
- **Contoh:** Diberikan $F = \{A \rightarrow C, AB \rightarrow CD\}$
  - C adalah ekstranus dalam $AB \rightarrow CD$ karena $AB \rightarrow C$ dapat disimpulkan bahkan setelah menghapus C.

Testing if an Attribute is Extraneous

Menguji Apakah suatu Atribut adalah Ekstranus

- Pertimbangkan sebuah himpunan F dari ketergantungan fungsional (*functional dependencies*) dan ketergantungan fungsional $\alpha \rightarrow \beta$ di dalam F.

- **Untuk menguji apakah atribut** $A \in \alpha$ **adalah ekstranus di dalam** α **:**
  - Hitung $(\{\alpha\} - A)^+$ menggunakan ketergantungan-ketergantungan yang ada di dalam F.
  - Periksa apakah $(\{\alpha\} - A)^+$ mengandung β; jika ya, maka A adalah **ekstranus** di dalam α.

- **Untuk menguji apakah atribut** $A \in \beta$ **adalah ekstranus di dalam** β**:**
  - Hitung α^+ hanya dengan menggunakan ketergantungan-ketergantungan di dalam $F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$,
  - Periksa apakah α^+ mengandung A; jika ya, maka A adalah **ekstranus** di dalam β.

Canonical cover

Penutup Kanonik (Canonical Cover)

- Sebuah **penutup kanonik** untuk F adalah sebuah himpunan ketergantungan F_c sedemikian rupa sehingga:
  - F secara logis menyiratkan semua ketergantungan dalam F_c, dan
  - F_c secara logis menyiratkan semua ketergantungan dalam F, dan
  - Tidak ada ketergantungan fungsional dalam F_c yang mengandung atribut ekstranus, dan
  - Setiap sisi kiri dari ketergantungan fungsional dalam F_c adalah unik (tidak ada duplikasi syarat).
- **Untuk menghitung penutup kanonik untuk F:**
  - **Ulangi (repeat):**
    - Gunakan aturan gabungan (*union rule*) untuk mengganti ketergantungan apa pun dalam F yang berbentuk :

      $$\alpha_1 \rightarrow \beta_1 \text{ dan } \alpha_1 \rightarrow \beta_2 \text{ dengan } \alpha_1 \rightarrow \beta_1 \beta_2$$
    - Temukan sebuah ketergantungan fungsional $\alpha \rightarrow \beta$ yang memiliki atribut ekstranus baik di dalam α maupun di dalam β.
      - **Catatan: pengujian atribut ekstranus dilakukan menggunakan F_c, bukan F.**
    - Jika ditemukan atribut ekstranus, hapus atribut tersebut dari $\alpha \rightarrow \beta$.
  - **Sampai (until)** F tidak berubah lagi.
- **Catatan:** Aturan gabungan (*union rule*) mungkin bisa diterapkan kembali setelah beberapa atribut ekstranus dihapus, jadi aturan ini harus terus diterapkan ulang.

Computing Canonical

- R = (A, B, C)
- *F* = {*A* → *BC*
  B → *C*
  A → *B*
  AB → *C*}
- **Gabungkan A → BC dan A → B menjadi A → BC**
  - Himpunan sekarang menjadi {A → BC, B → C, AB → C}
- **A adalah ekstranus dalam AB → C**
  - Periksa apakah hasil dari menghapus A dari AB → C (yaitu B → C) tersirat oleh ketergantungan lainnya.
    - **Ya:** bahkan, B → C memang sudah ada di dalam himpunan!
  - Himpunan sekarang menjadi {A → BC, B → C}
- **C adalah ekstranus dalam A → BC**
  - Periksa apakah A → C secara logis tersirat oleh A → B dan ketergantungan lainnya.
    - **Ya:** menggunakan aturan transitif pada A → B dan B → C.
    - Dapat menggunakan *attribute closure* dari A pada kasus-kasus yang lebih kompleks.
- **Penutup Kanonik (*Canonical Cover*) adalah:**
  A → B
  B → C

Bentuk Normal Kedua (2NF)

- Sebuah relasi berada dalam **2NF** jika dan hanya jika relasi tersebut berada dalam **1NF** dan setiap atribut yang bukan kunci (***nonkey attribute***) **bergantung** secara penuh (***irreducibly dependent***) pada kunci primer (***primary key***).
- 2NF masih memperbolehkan adanya **ketergantungan transitif** (*transitive dependencies*).
- Ketergantungan transitif dapat menyebabkan **anomali pembaruan** (*update anomalies*).

Tujuan Normalisasi

- Anggaplah R sebagai skema relasi dengan himpunan ketergantungan fungsional F.
- Tentukan apakah skema relasi R berada dalam bentuk yang "baik" (*good form*).
- Dalam hal skema relasi R tidak berada dalam bentuk yang "baik", **pecahlah** (*decompose*) skema tersebut ke dalam satu **set skema relasi** $\{R_1, R_2, \ldots, R_n\}$ sedemikian rupa sehingga:
  - Masing-masing skema relasi berada dalam bentuk yang "baik".
  - Dekomposisi tersebut merupakan **lossless-join decomposition** (artinya: data tidak hilang saat tabel-tabel digabungkan kembali).
  - Sebaiknya, dekomposisi tersebut harus **dependency preserving** (artinya: ketergantungan antar data tetap terjaga/tidak hilang setelah pemecahan tabel).

Dekomposisi Gabungan Tanpa Kehilangan (*Lossless-join*)

- Untuk kasus $R = (R_1, R_2)$, kita mensyaratkan bahwa untuk semua kemungkinan relasi r pada skema R:

$$r = \Pi_{R1}(r) \bowtie \Pi_{R2}(r)$$

- Sebuah dekomposisi dari R menjadi R_1 dan R_2 adalah **lossless join** (tanpa kehilangan data saat digabung) jika setidaknya salah satu dari ketergantungan berikut ada dalam F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$

Contoh

- $R = (A, B, C)$
  $F = \{A \rightarrow B, B \rightarrow C)$
  - Dapat didekomposisi (dipecah) dengan dua cara berbeda:
- **Cara 1:** $R_1 = (A, B)$, $R_2 = (B, C)$
  - **Lossless-join decomposition:**
    $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$.
  - **Dependency preserving** (Menjaga ketergantungan): Semua aturan asli
    ($A \rightarrow B$ *dan* $B \rightarrow C$)
    masih bisa dicek langsung di masing-masing tabel hasil pecahannya.
- **Cara 2:** $R_1 = (A, B)$, $R_2 = (A, C)$
  - **Lossless-join decomposition:**
    $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
  - **Not dependency preserving** (Tidak menjaga ketergantungan): Kita tidak bisa mengecek aturan $B \rightarrow C$ tanpa menggabungkan kembali tabel R_1 dan R_2

Pemeliharaan Ketergantungan (*Dependency Preservation*)

- Misalkan F_i adalah himpunan ketergantungan di dalam F^+ yang hanya mencakup atribut-atribut dalam R_i.
  - Sebuah dekomposisi disebut **dependency preserving** (memelihara ketergantungan), jika:

$$(F_1 \cup F_2 \cup \ldots \cup F_n)^+ = F^+$$

  - **Jika tidak**, maka pengecekan pembaruan (*updates*) terhadap pelanggaran ketergantungan fungsional mungkin memerlukan perhitungan gabungan (*join*), yang biayanya sangat mahal (memakan sumber daya komputasi yang tinggi).

Testing for Dependency Preservation

- Untuk memeriksa apakah suatu ketergantungan $\alpha \rightarrow \beta$ terjaga (***preserved***) dalam dekomposisi R menjadi $R_1, R_2, \ldots, R_n$, kita **menerapkan pengujian berikut** (dengan penutupan atribut atau *attribute closure* yang dilakukan terhadap F):
  - *result* = α
    while (ada perubahan pada *result*) **do**
    for each R_i dalam dekomposisi:
    $t = (result \cap R_i)^+ \cap R_i$
    $result = result \cup t$
  - Jika *result* mengandung semua atribut dalam β, maka ketergantungan fungsional $\alpha \rightarrow \beta$ dinyatakan **terjaga** (*preserved*).
- Kita menerapkan pengujian ini pada semua ketergantungan dalam F untuk memeriksa apakah suatu dekomposisi bersifat memelihara ketergantungan (*dependency preserving*).
- Prosedur ini memakan waktu polinomial, alih-alih waktu eksponensial yang diperlukan untuk menghitung F^+ dan $(F_1 \cup F_2 \cup \ldots \cup F_n)^+$

Contoh

- $R = (A, B, C)$
  $F = \{A \rightarrow B$
  $B \rightarrow C\}$
  Key = $\{A\}$

- **R tidak berada dalam BCNF** (Boyce-Codd Normal Form)
- **Dekomposisi** $R_1 = (A, B)$, $R_2 = (B, C)$
  - R_1 dan R_2 berada dalam **BCNF**.
  - Merupakan **Lossless-join decomposition** (Gabungan tanpa kehilangan data).
  - Bersifat **Dependency preserving** (Menjaga ketergantungan).

Arigatou Gozaimasu